CineChef

La noche de peli perfecta

Mini-proyecto DP-800 · Nivel 3 (Avanzado)

SQL Server 2025 · embeddings locales · búsqueda híbrida · RAG

Autora

Noelia Segura Herrero

Repositorio

github.com/noeliaseguraherrero/cinechef-dp800

Octubre de 2026 · Trabajo individual

CONTENIDO

01 Objetivo del proyecto
02 Caso de uso
03 Arquitectura
04 Modelo de datos
05 Funcionalidades implementadas
06 Consultas y código más importante
07 Seguridad y optimización
08 Funcionamiento de la parte de IA
09 Capturas de resultados
10 Problemas encontrados
11 Conclusiones

APARTADO 1

Objetivo del proyecto

CineChef responde a una pregunta cotidiana: **¿qué cenamos y qué vemos esta noche?** El proyecto construye, sobre SQL Server 2025 y sin salir del motor, un sistema que recomienda **una película y una receta que casen entre sí** a partir de una descripción libre del estado de ánimo, y que lo hace de forma **segura**: respeta las alergias del usuario y la edad mínima de cada película.

Qué se quería demostrar	Cómo
Diseñar y proteger	Modelo relacional, vistas, funciones, procedimientos y trigger (Bloque A); roles, Row-Level Security, enmascarado y auditoría (Bloque C)
Buscar con IA	Embeddings generados en local, búsqueda vectorial, Full-Text y búsqueda híbrida (Bloques E y F)
Generar con contexto	RAG con GitHub Copilot y verificación de la respuesta en SQL (Bloque G)
Medir	Optimización con lecturas lógicas antes y después (Bloque D) y desarrollo asistido por Copilot documentado (Bloque B)

Nivel elegido: **3 (Avanzado)**, que combina los bloques A a G. Trabajo individual y todo en local, sin Azure ni Fabric.

APARTADO 2

Caso de uso

Un usuario de **58 años que evita el gluten y la leche** escribe cómo se siente. El sistema recupera solo lo que es seguro para él y deja que el modelo redacte la recomendación con ese material.

Paso	Qué ocurre
1. Pregunta	*a comforting dinner for a rainy night to watch with friends* (en inglés: el modelo de embeddings trabaja en ese idioma)
2. Filtro en SQL	Solo recetas sin gluten ni leche y películas aptas para su edad, **antes** de que ningún modelo vea datos
3. Búsqueda	La búsqueda híbrida (vectores + Full-Text) devuelve 3 recetas y 3 películas
4. Generación	SQL monta el prompt con ese contexto; Copilot Chat redacta la respuesta (paso manual)
5. Verificación	SQL comprueba que solo se citan elementos recuperados, sin alérgenos y con edad apta

Resultado: *sopa de gambas, coco y lima* [R80] junto a *El apartamento* [M5]. La verificación en SQL da OK.

Con un usuario de 10 años, el filtro de edad descarta el terror antes de generar nada: Copilot propone *Spirited Away* [M8].

APARTADO 3

Arquitectura

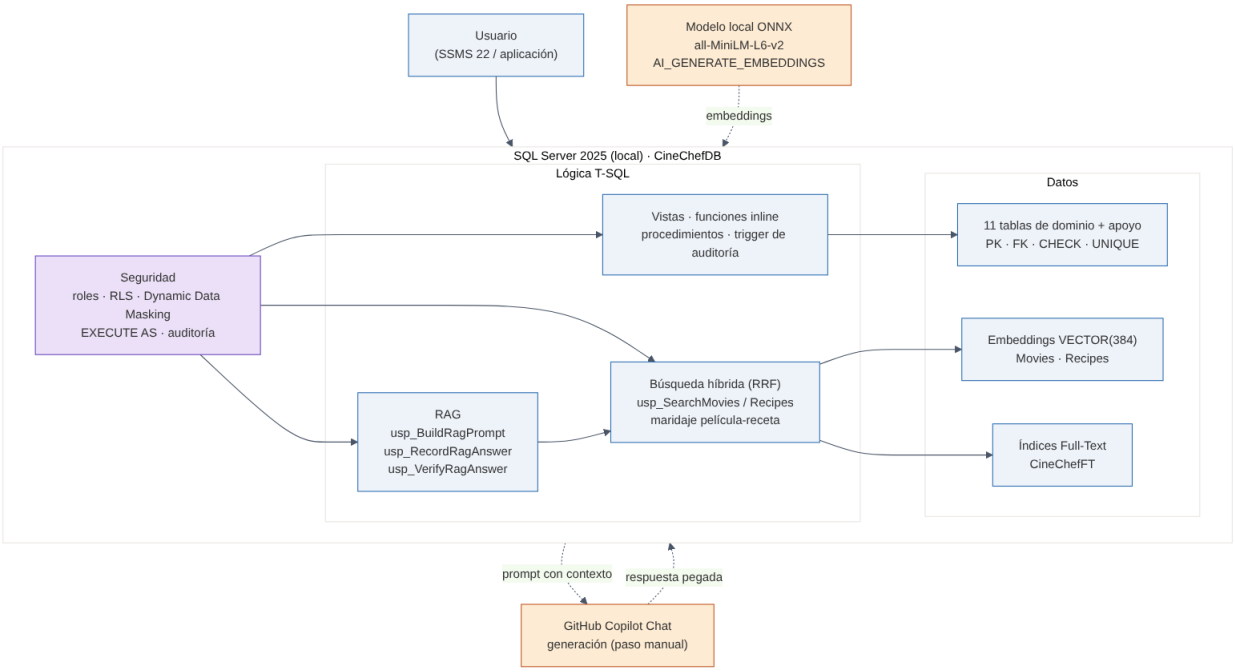


Figura 1. Arquitectura: datos, lógica y seguridad viven en SQL Server; solo el modelo ONNX (local) y Copilot (paso manual) son externos.

- **SQL Server 2025 local** contiene el catálogo, la lógica T-SQL (vistas, funciones, procedimientos, trigger) y la capa de seguridad.
- **Modelo ONNX local** (all-MiniLM-L6-v2, 384 dimensiones): genera los embeddings dentro del motor con **AI_GENERATE_EMBEDDINGS**.
- **Búsqueda híbrida:** combina el ranking vectorial y el de Full-Text con Reciprocal Rank Fusion.
- **GitHub Copilot Chat** redacta la respuesta del RAG. No se puede invocar desde T-SQL, así que es un paso manual y documentado.

Idea de diseño: la seguridad se aplica en SQL **antes** de que el modelo reciba datos, y la respuesta se verifica **después** contra la base de datos. El modelo de lenguaje no es la fuente de verdad: lo es el catálogo.

APARTADO 4

Modelo de datos

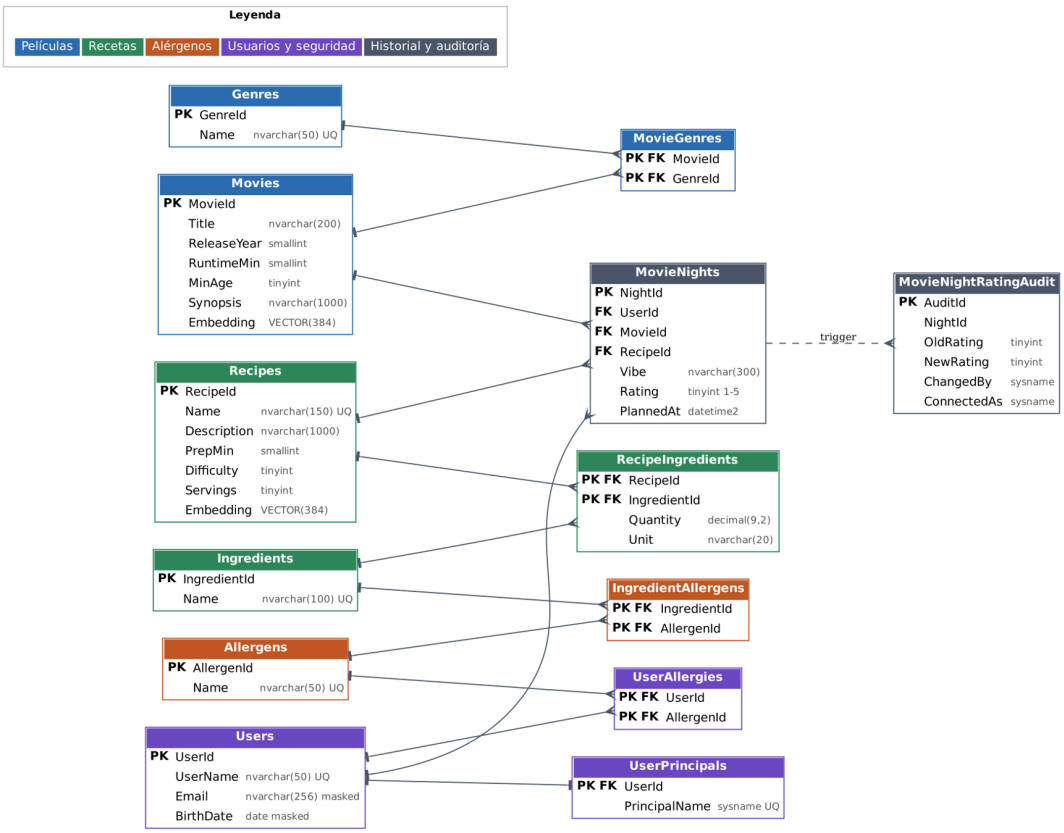


Figura 2. Modelo de datos: 11 tablas de dominio y 4 de apoyo (identidad, auditoría, ejecuciones de embeddings y sesiones RAG).

Grupo	Tablas (filas)
Películas	Genres (18) · Movies (100) · MovieGenres (233)
Recetas	Ingredients (120) · Recipes (100) · RecipeIngredients (500)
Alérgenos	Allergens (14) · IngredientAllergens (39)
Usuarios	Users (10) · UserAllergies (9) · MovieNights (30)
Apoyo	UserPrincipals · MovieNightRatingAudit · EmbeddingRun · RagSession

Restricciones: clave primaria en todas las tablas (compuesta en las de relación), claves foráneas, **UNIQUE** en nombres y **CHECK** en edad mínima (0, 7, 12, 16, 18), valoración (1 a 5) y fecha de nacimiento no futura. *Movies* y *Recipes* incluyen **Embedding VECTOR(384)**. Los datos son ficticios.

APARTADO 5

Funcionalidades implementadas

Bloque	Qué se implementó	Pruebas
A Diseño y T-SQL	4 vistas, 2 funciones inline (recetas seguras y películas permitidas), procedimientos para registrar y valorar planes con control de errores, búsqueda difusa por título, trigger de auditoría, CTE recursiva y consultas avanzadas	32 + 16 + 7
B Copilot	7 interacciones documentadas (datos de prueba y revisión de código), cada una con prompt, respuesta, partes aceptadas y corregidas y verificación	fichas B-01 a B-07
C Seguridad	Roles con mínimo privilegio, Row-Level Security, Dynamic Data Masking, EXECUTE AS y auditoría de valoraciones	35
D Optimización	Índice sobre las recetas por ingrediente, medición con 500 000 filas y experimento con índice vectorial	Q2: 16,5×
E Embeddings	Modelo ONNX local, generación de 384 dimensiones, comparación de formatos de texto	8
F Búsqueda	Búsqueda vectorial, Full-Text e híbrida (RRF), con filtro por alergias y edad, y maridaje película-receta	10
G RAG	Prompt construido en SQL, registro de la respuesta y verificación automática; 3 preguntas con y sin contexto	9

A esto se suman 8 pruebas de humo del entorno: **125 pruebas automáticas en OK** en total. Todas siguen el mismo patrón: una tabla temporal con OK o REVISAR y un error final si alguna falla, de modo que ningún script puede terminar "en verde" con un fallo dentro.

La carga completa se puede repetir desde cero con un solo comando (**scripts/load_all.ps1**), y la reconstrucción desde cero se ejecutó con resultado correcto.

APARTADO 6

Consultas y código más importante

Tres fragmentos resumen la idea del proyecto. El código completo está en el repositorio.

Recetas seguras: la función que se "cierra" ante la duda

```
CREATE FUNCTION dbo.fn_SafeRecipes (@UserId INT) RETURNS TABLE AS RETURN
SELECT r.RecipeId, r.Name, r.Description
FROM dbo.Recipes AS r
WHERE EXISTS (SELECT 1 FROM dbo.Users WHERE UserId = @UserId)             -- usuario existente
  AND EXISTS (SELECT 1 FROM dbo.RecipeIngredients WHERE RecipeId = r.RecipeId) -- con ingredientes
  AND NOT EXISTS (SELECT 1 FROM dbo.RecipeIngredients ri                  -- sin sus alérgenos
        JOIN dbo.IngredientAllergens ia ON ia.IngredientId = ri.IngredientId
        JOIN dbo.UserAllergies ua ON ua.AllergenId = ia.AllergenId AND ua.UserId = @UserId
        WHERE ri.RecipeId = r.RecipeId);
```

Búsqueda híbrida: fusión de dos rankings (RRF)

```
score = 1.0 / (@RrfK + v.rnk)                      -- posición en la lista vectorial
      + COALESCE(1.0 / (@RrfK + t.rnk), 0)        -- posición en la lista Full-Text   (@RrfK = 60)
```

Row-Level Security: cada persona ve solo sus filas

```
CREATE SECURITY POLICY Security.pol_UserData
  ADD FILTER PREDICATE Security.fn_OwnerOrAdmin(UserId) ON dbo.MovieNights,
  ADD BLOCK  PREDICATE Security.fn_OwnerOrAdmin(UserId) ON dbo.MovieNights AFTER INSERT,
  ADD BLOCK  PREDICATE Security.fn_OwnerOrAdmin(UserId) ON dbo.MovieNights AFTER UPDATE
  WITH (STATE = ON);   -- (extracto: la política también cubre Users y UserAllergies)
```

Los embeddings se generan en el propio motor con **AI_GENERATE_EMBEDDINGS(texto USE MODEL CineChefEmbeddingModel)**, y la búsqueda exacta ordena por **VECTOR_DISTANCE('cosine', Embedding, @consulta)**. Otros objetos importantes: **usp_SearchRecipes**, **usp_BuildRagPrompt** y **usp_VerifyRagAnswer**.

APARTADO 7

Seguridad y optimización

Seguridad

Mecanismo	Qué consigue
Roles y permisos	Mínimo privilegio: los clientes solo ejecutan procedimientos, el analista no lee datos personales
Row-Level Security	Cada cliente ve y escribe solo sus propias filas (usuarios, alergias, planes)
Dynamic Data Masking	El nutricionista ve el email y la fecha de nacimiento enmascarados
EXECUTE AS OWNER	El analista obtiene estadísticas por género (mínimo 3 valoraciones) sin leer los planes
Auditoría	Un trigger registra cada cambio de valoración con el usuario real que lo hizo

Limitación demostrada: el enmascarado **no es control de acceso**. Con permiso de lectura sobre la columna, el nutricionista acertó la fecha filtrando por el valor real; con **DENY SELECT** el mismo ataque falla (error 230). La mitigación se probó pero no se aplicó, y el riesgo queda documentado.

Optimización

Consulta (500 000 filas)	Lecturas antes	Lecturas después	Mejora
Q1. Recetas con un ingrediente (TOP 20)	22	11	2×
Q2. Recetas con algún alérgeno (COUNT DISTINCT)	1 859	113	**16,5×**
Q3. 20 primeras recetas seguras de un usuario	1 147	1 147	sin cambio

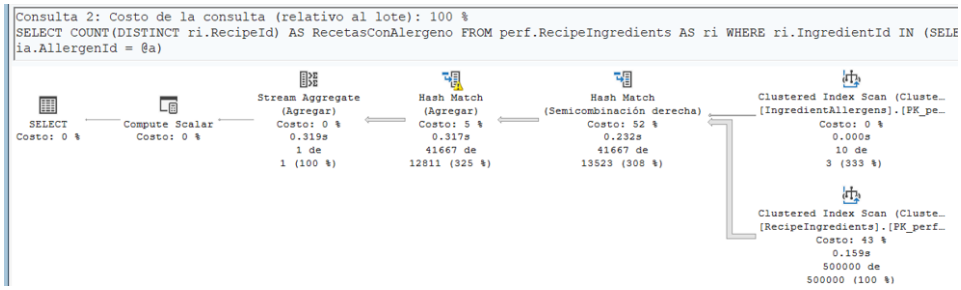


Figura 3a. Plan de Q2 sin índice: Clustered Index Scan sobre RecipeIngredients (500 000 filas).

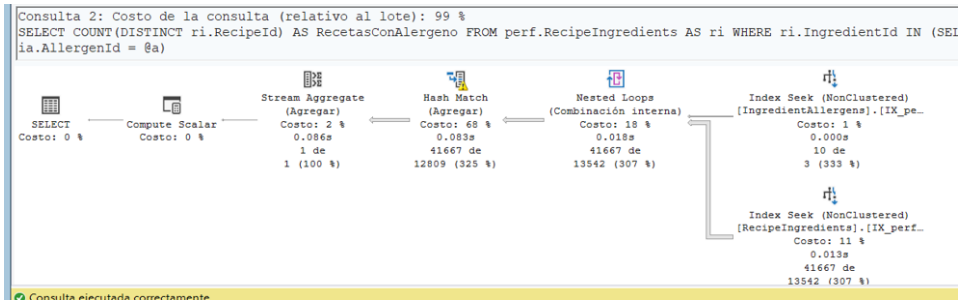


Figura 3b. Plan de Q2 con índices: Index Seek sobre los dos índices nuevos.

Se creó el índice sobre *RecipeIngredients(IngredientId)* en el catálogo real. Un índice vectorial DiskANN (vista previa) acierta 10 de 11 vecinos y tarda menos (5 a 13 ms frente a 21 a 33 ms), pero lee más páginas y deja la tabla de solo lectura; con 100 elementos se usa la búsqueda exacta.

APARTADO 8

Funcionamiento de la parte de IA

Embeddings. Cada película y receta se convierte en un vector de 384 dimensiones con el modelo ONNX local, sin salir del motor. Se comparó el formato del texto de entrada: con etiqueta de géneros ganó en películas (MRR 0,334) y sin ella en recetas (MRR 0,692). La calidad es modesta y el modelo no entiende español, así que el catálogo y las consultas van en inglés.

Búsqueda. Tres modos: vectorial, Full-Text e híbrido (RRF). Con las sondas de este catálogo el texto gana y el híbrido no lo mejora; el vector aporta como respaldo y en el maridaje película-receta, donde no hay vocabulario común.

Sondas (recetas)	Texto	Híbrido	Vector
Léxicas, 20 · acierto en 1.ª posición	19	18	12
Semánticas, 8 · MRR	0,490	0,352	0,128

RAG. SQL recupera 3 recetas y 3 películas ya filtradas por alergias y edad, construye el prompt, Copilot responde y SQL verifica que solo se cita lo recuperado, sin alérgenos y con edad apta.

Pregunta	Con RAG	Sin RAG
1. Cena reconfortante, 58 años	Sopa de gambas [R80] + *El apartamento* [M5] → **OK**	*Chef* y un curry "sin gluten" que no están en el catálogo → REVISAR
2. Noche de miedo, 10 años	*Spirited Away* [M8] + risotto [R11] → **OK**	*Coraline* y pizza momia → REVISAR
3. Ramen y película de samuráis	*Seven Samurai* [M17] + cerdo al romero [R24]; avisa de que no hay ramen → **OK**	*Los siete samuráis* y un ramen sin gluten → REVISAR

Qué aporta el RAG: no mejora la redacción, aporta **trazabilidad y seguridad**. Cada elemento existe en la base de datos con sus alérgenos y su edad mínima, y se puede comprobar; sin contexto, lo de "sin gluten" es solo la palabra del modelo. Las primeras líneas base salieron contaminadas por la conversación y se repitieron en chats nuevos. Son solo 3 preguntas y un modelo: es una demostración, no una evaluación estadística.

APARTADO 9

Capturas de resultados

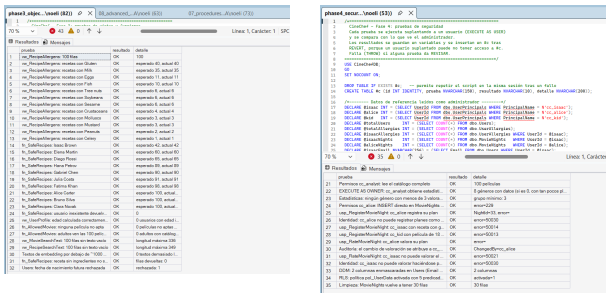


Figura 4. Pruebas automáticas en OK: vistas y funciones (32, izquierda) y seguridad (35, derecha).



Figura 5. RAG con el prompt generado en SQL Server. Izquierda, pregunta 1 (58 años, sin gluten ni leche): *sopa de gambas* [R80] y *El apartamento* [M5]. Derecha, pregunta 3 (ramen y película de samuráis): *Seven Samurai* [M17] y cerdo al romero [R24], avisando de que no hay ramen.

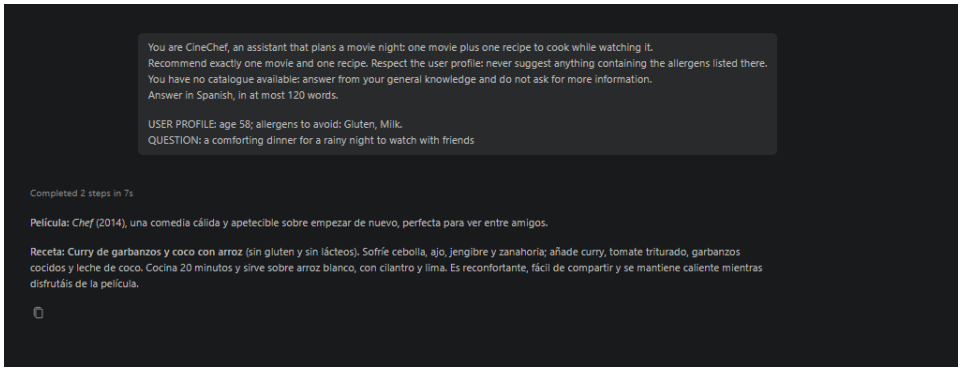


Figura 6. La misma pregunta 1 sin contexto (línea base): Copilot inventa *Chef* y un curry "sin gluten" que no están en el catálogo y que SQL no puede verificar.

Las capturas completas, con todos los resultados y también los errores, están en **docs/evidencias/** con un índice.

APARTADO 10

Problemas encontrados

Problema	Causa	Solución
Error 31739 al generar embeddings	El tokenizador se llamaba *tokenizers_cpp-release.dll* en lugar de *tokenizers_cpp.dll*	Renombrar el archivo
Error 8116 con *AI_GENERATE_EMBEDDINGS* dentro de *VECTOR_DISTANCE*	La función solo devuelve vector al asignarse a una columna o variable	Tabla temporal con columna VECTOR(384)
La búsqueda en español no funciona	El modelo se entrenó sobre todo con inglés	Catálogo y consultas en inglés
Distancias coseno muy comprimidas (0,12 a 0,14)	El modelo separa poco los resultados	Fusión por posiciones (RRF) y Full-Text
La auditoría atribuía los cambios a *sa*	Solo se guardaba el inicio de sesión	Guardar también el usuario de base de datos
El enmascarado no impide deducir el valor	No es control de acceso	Demostrado; *DENY SELECT* lo bloquea
Líneas base del RAG contaminadas	La conversación ya había visto el contexto	Repetirlas en chats nuevos, modo Ask
Índice vectorial: tabla de solo lectura (42231)	Función en vista previa	Búsqueda exacta con 100 elementos

Hay 17 entradas con su detalle en **docs/problemas_encontrados.md**.

APARTADO 11

Conclusiones

- **Lo que funcionó:** aplicar la seguridad en SQL antes del modelo, verificar el RAG contra la base de datos, 125 pruebas en OK y una mejora de 16,5× en la consulta de alérgenos.
- **Lo que salió peor de lo esperado:** la calidad semántica del modelo local es modesta, la búsqueda vectorial sola no supera al texto, y el RAG no mejora la redacción frente a un LLM solo.
- **IA asistida:** Copilot acelera los datos y la búsqueda de casos límite, pero nunca ejecutó el código; cada afirmación se aceptó solo con una prueba ejecutable.
- **Mejoras posibles:** un modelo multilingüe, separar la pregunta en parte de receta y de película, salida estructurada con ids y SQL Server Audit.